LAPORAN TUGAS BESAR 1: *SEARCHING*
IF3170 INTELIGENSI ARTIFISIAL
SEMESTER 1 2026/2027



Oleh

Kelompok LocalKirkification (39)

Naufal Baldemar Ardanni	13521154
Reynard Anderson Wijaya	13524111
Rainaldi Pratama F Sembiring	13524117

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Oktober 2026

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN....2
I.1 Deskripsi....2
I.2 Spesifikasi....2
BAB II PEMBAHASAN....5
II.1 Penjelasan Implementasi Algoritma Local Search....5
II.1.1 Representasi State....5
II.1.2 State Awal....5
II.1.3 Constraint....6
II.1.4 Fungsi Objektif....6
II.1.5 Pembangkitkan Neighbor....7
II.2 Penjelasan Implementasi Algoritma Hill Climbing: Steepest Ascent....7
II.3 Penjelasan Implementasi Algoritma Simulated Annealing....8
II.4 Penjelasan Implementasi Algoritma Genetic Algorithm....9
II.5 Hasil dan Analisis Eksperimen....10
II.5.1 Lingkungan dan Skenario Pengujian....10
II.5.2 Hasil Pengujian Hill Climbing....11
II.5.3 Hasil Pengujian Simulated Annealing....13
II.5.4 Hasil Pengujian Genetic Alorithm....15
II.5.5 Perbandingan Algoritma....17
BAB III KESIMPULAN DAN SARAN....20
III.1 Kesimpulan....20
III.2 Saran....21
BAB IV PEMBAGIAN TUGAS....23

BAB I

DESKRIPSI PERSOALAN

I.1 Deskripsi

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi *driver* truk uji coba kepada Special Week karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, *toreena-toreena* yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

I.2 Spesifikasi

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti memaksimalkan fungsi objektif.

Tabel I.1 Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu z.

Entitas	Atribut	Contoh	Keterangan
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	*Estimated Time of Arrival*, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus/balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar)
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (*initial state*) random.
- Langkah *neighbor* (*move*) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu x, y, atau z
- Representasi solusi (*final state*) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh *state* dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar

2. Bagaimana keadaan peti kemas pada truk
3. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan.

- ***Objective function***

  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.

- **Kardus tidak bisa melayang dan menembus dinding peti kemas**

  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(xi, yi, zi)$$

  dan dimensi

  $$(wi, di, hi)$$

  sehingga alas paket i adalah

  $$Bi = \{(x,y,zi) \mid xi \ x < xi+wi, yi \ y < yi+di \})$$

  Sebuah titik di alas memiliki *support* apabila

  1. Zi = 0, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Melayang adalah kondisi ketika tidak ada satu pun titik alas yang memiliki *support*.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*.

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

BAB II

PEMBAHASAN

II.1 Penjelasan Implementasi Algoritma *Local Search*

Pada tugas ini, ketiga algoritma search (*Steepest Ascent*, *Simulated Annealing*, dan *Genetic Algorithm*) diimplementasikan menggunakan komponen berupa: representasi state, constraint, fungsi objektif, dan pembangkitkan neighbour.

II.1.1 Representasi State

Paket direpresentasikan oleh dataclass Package (id, dims, value, weight, isFragile, eta, orient) dan truk oleh Truck (dims, maxCapacity). State dikelola oleh kelas State dengan tiga komponen:

- packages: daftar paket, termasuk dengan orientasi masing-masing.
- truck: truk yang dipakai
- pos: daftar posisi, dengan pos[i] = (x,y,z) adalah koordinat sudut paket i yang minimum, atau None jika paket di luar truk.

Orientasi disimpan sebagai permutasi indeks pada atribut orient. Sehingga ukuran efektif paket pada sumbu x, y, z diperoleh dari dims (w,l,h) yang diurutkan orient. Operasi rotate(i, axis) menukar dua elemen orient. Satu paket di dalam truk menempati balok dari (x,y,z) sampai (x+w,y+l,z+h). Method copy() membuat salinan state agar pembangkitan neighbor tidak mengubah state asal. atribut eta disimpan pada paket tetapi tidak digunakan dalam constraint maupun fungsi objektif.

Constraint tidak dijamin terpenuhi saat state dibangkitkan, maka state yang melanggar aturan tetap boleh ada, namun dikenai penalti besar pada fungsi objektif sehingga pencarian dapat melewati state yang tidak valid.

II.1.2 State Awal

State awal dibangkitkan oleh State.random_init dengan p_inside = 0,5. Tiap paket diberi orientasi acak. Dengan peluang 0,5 dan bila ukurannya setelah

rotasi muat dalam dimensi truk, paket ditempatkan pada koordinat bulat acak di dalam batas truk, paket lainnya berada di luar. Overlap dan paket melayang tidak dicek saat inisialisasi, sehingga state awal umumnya tidak valid dengan skor negatif.

II.1.3 Constraint

Fungsi violations(s) di constraints.py menghitung jumlah pelanggaran per jenis, dan state valid apabila pelanggarannya nol (is_valid). Jenis pelanggarannya:

- out_of_bounds: balok paket menembus dinding truk pada salah satu sumbu.
- overlap: dua paket di dalam truk beririsan volumenya. Dihitung satu pelanggaran per pasangan paket.
- floating: paket yang tidak berada di alas (z > 0) dan tidak memiliki penyangga. Penyangga adalah paket lain yang sisi atasnya tepat berada pada z paket tersebut dan alasnya beririsan luas dengan alas paket (di bidang x-y). Dihitung satu pelanggaran per paket yang melayang.
- fragile_support: paket pecah-belah yang menjadi penyangga paket lain. Dihitung per pasangan, yaitu satu pelanggaran untuk setiap paket pecah-belah yang menopang suatu paket.
- overweight: total berat paket di dalam truk melebihi maxCapacity, nilainya 1 bila terlanggar.

II.1.4 Fungsi Objektif

Fungsi objektif (objective.py) menghitung:

$$score(s) = inside_value(s) - penalty\ x\ total_violations(s)$$

Di mana inside_value adalah jumlah nilai semua paket yang posisinya bukan None (di dalam truk), total_violations adalah jumlah seluruh hitungan dari violations(s), dan penalty adalah 1000. Nilai penalti diatur 1000 (jauh lebih besar dari nilai satu paket) sehingga melanggar aturan tidak pernah menguntungkan. Maka, state valid adalah state yang skornya sama dengan total nilai paket di dalam truk.

II.1.5 Pembangkitkan Neighbor

Neighbor dibangkitkan oleh neighbors.py dengan tiga jenis perubahan pada copy state:

- swap: menukar posisi dua paket. Orientasi tetap sesuai paketnya, sehingga setelah penukaran paket dapat melanggar batas atau tumpang tindih dan dinilai oleh penalti.
- rotate: merotasi satu paket pada salah satu dari tiga sumbu.
- move: memindahkan satu paket. Jika paket berada di dalam truk dan tidak muat atau terpilih dengan peluang 0,2, paket dikeluarkan dari truk. Selain itu, paket dipindahkan ke koordinat bulat acak di dalam batas truk. Pencarian posisi baru diulang hingga 10 kali sampai berbeda dari posisi lama. Paket yang tidak muat dan sudah di luar tidak diubah.

Ada dua cara pemakaian. random_neighbor memilih satu dari tiga jenis perubahan secara acak dengan peluang sama. Fungsi ini dipakai Simulated Annealing (satu neighbor per iterasi) dan Genetic Algorithm (sebagai operator mutasi). all_neighbors menyusun kumuplan neighbor yang terdiri atas seluruh kombinasi swap, seluruh rotasi, dan k_move sampel move, karena ruang koordinat terlalu besar untuk dienumerasi semua. Fungsi ini hanya dipakai Hill Climbing.

II.2 Penjelasan Implementasi Algoritma *Hill Climbing: Steepest Ascent*

Hill Climbing steepest ascent pada setiap iterasi mengevaluasi seluruh neighbor dari state saat ini, lalu berpindah ke neighbor dengan skornya (state value) tertinggi asalkan skornya lebih baik dari state saat ini. Implementasinya ada pada fungsi steepest_ascent(init, max_iter=1000, k_move=100).

Alur algoritma:

1. State awal disalin, skornya dihitung dan dicatat sebagai elemen pertama riwayat.
2. Ulangi paling banyak max_iter kali:
   a. Bangkitkan neighbor dengan all_neighbors(cur, k_move)

b. Hitung skor tiap neighbor dan simpan neighbor dengan skor tertinggi yang lebih besar daripada skor saat ini.

c. Jika tidak ada neighbor yang lebih baik, berhenti. Jika ada, pindah ke neighbor tersebut dan catat skornya ke history.

3. Kembalikan state awal dan akhir, skor akhir, history skor, jumlah iterasi, dan durasi.

Kondisi berhenti ada dua: tidak ada neighbor dengan skor lebih tinggi (local optimum), atau iterasi mencapai batas 1000. Karena perbandingan bersifat ketat, neighbor dengan skor sama tidak diterima.

II.3 Penjelasan Implementasi Algoritma *Simulated Annealing*

Simulated Annealing membangkitkan satu neighbor acak pada tiap iterasi. Neighbor yang sama atau lebih baik selalu diterima, sedangkan neighbor yang lebih buruk diterima dengan peluang e^(ΔE/T) yang mengecil seiring turunnya suhu, sehingga pencarian dapat keluar dari local optimum. Implementasinya ada pada simulated_annealing(init, t0, alpha=0.999, max_iter=20000, stuck_limit=200).

Alur algoritma:

1. State awal disalin, suhu T diset ke t0, dan state terbaik diinisialisasi dengan state awal.
2. Ulangi max_iter kali:

   a. Ambil satu neighbor acak dengan random_neighbor, lalu hitung ΔE = skor neighbor - skor saat ini.

   b. Hitung peluang p = exp(min(ΔE / T, 0)). Bila ΔE >= 0, p = 1 (termasuk ΔE = 0, yaitu perpindahan menyamping diterima), dan bila ΔE < 0, p = e^(ΔE/T).

   c. Terima neighbor bila bilangan acak antara 0 dan 1 lebih kecil dari p. Bila diterima dan skornya melampaui best, perbarui best.

   d. Perbarui penghitung no_gain: direset ketika langkah yang diterima memperbaiki skor (ΔE > 0), dan bertambah pada kasus lain (langkah ditolak atau diterima tanpa perbaikan). Ketika no_gain mencapai

stuck_limit (200), stuck_count bertambah satu, dan dihitung sekali sampai ada perbaikan berikutnya.

e. Turunkan suhu secara geometrik: T = max(T x alpha, 1e-9).

3. Kembalikan state terbaik beserta riwayat dan statistik.

Parameter yang dipakai pada eksperimen adalah t0 = 1000 (nilai bawaan fungsi 100), alpha = 0.999, max_iter = 20000, dan stuck_limit = 200. Nilai t0 sebanding dengan penalti satu pelanggaran, sehingga akan toleran ketika menerima pilihan baru yang mempunyai satu pelanggaran.

II.4 Penjelasan Implementasi Algoritma *Genetic Algorithm*

Genetic Algorithm mempertahankan populasi solusi, memilih individu (state) yang baik sebagai parent, menggabungkan dengan crossover, dan memutasi sebagian anak untuk membentuk generasi berikutnya. Implementasinya ada pada genetic_algorithm(packages, truck, pop_size=50, generations=100, p_mut=0.3, k_tour=3).

Alur algoritma:

1. Bangkitkan populasi awal sebanyak pop_size state acak dengan State.random_init, hitung fitness, dan simpan individu terbaik sebagai state awal yang dilaporkan.
2. Untuk tiap generasi:|

   a. Catat skor maksimum dan rata-rata populasi.

   b. Salinan individu terbaik langsung dimasukkan ke generasi baru.

   c. Selama generasi baru belum penuh, pilih dua parent dengan tournament selection (ambil k_tour = 3 individu acak, pilih fitness-nya tertinggi, dilakukan dua kali secara independen sehingga kedua parent bisa sama), lakukan uniform crossover, lalu dengan peluang p_mut = 0,3 mutasi anak dengan satu langkah random_neighbor (swap, move, atau rotate).

   d. Ganti seluruh populasi dengan generasi baru, hitung ulang fitness, dan perbarui solusi terbaik sepanjang pencarian jika ada skor yang lebih tinggi.
3. Setelah generasi terakhir, catat statistik akhir dan kembalikan solusi terbaik.

Uniform crossover membuat anak dari salinan parent pertama, lalu untuk tiap paket dengan peluang 0,5 posisi dan orientasi paket diambil dari parent kedua. Keduanya diambil bersamaan agar orientasi tetap konsisten. Anak tidak diperbaiki, jadi pelanggaran yang muncul dibiarkan dihukum oleh penalti.

II.5 Hasil dan Analisis Eksperimen

II.5.1 Lingkungan dan Skenario Pengujian

Eksperimen menggunakan data yang terdiri atas 30 paket dengan dimensi, berat, nilai, status kerapuhan, dan ETA yang berbeda. Truk yang digunakan memiliki dimensi 10 × 20 × 5 satuan dengan kapasitas maksimum 200 satuan berat.

Seluruh algoritma menggunakan fungsi objektif dan pemeriksaan constraint yang sama sehingga hasilnya dapat dibandingkan berdasarkan kriteria evaluasi yang seragam.

Pengujian dilakukan sebanyak tiga kali untuk setiap konfigurasi dengan pengaturan seed bilangan acak. Pada Hill Climbing dan Simulated Annealing, state awal yang digunakan sama untuk setiap pasangan pengujian, meskipun proses pembangkitan neighbor menggunakan urutan bilangan acak yang berbeda.

Tabel II.1 Tabel Parameter Algoritma

Algoritma	Parameter	Nilai
Hill Climbing	max iter	1000
Hill Climbing	k_move	100
simulated annealing	initial temperature	1000
simulated annealing	alpha	0.999
simulated annealing	max_iter	20000
simulated annealing	stuck_limit	200
genetic algorithm	population size	20,50,100
genetic algorithm	generation size	50,100,200

genetic algorithm	mutation probability	0,3
genetic algorithm	tournament size	3

Khusus untuk Genetic Algorithm, eksperimen dibagi menjadi dua kelompok pengujian. Kelompok pertama menggunakan ukuran populasi tetap sebesar 50 individu dengan variasi jumlah generasi sebesar 50, 100, dan 200. Pengujian ini bertujuan untuk mengamati pengaruh jumlah generasi terhadap kualitas solusi dan waktu eksekusi.

Kelompok kedua menggunakan jumlah generasi tetap sebesar 100 dengan variasi ukuran populasi sebesar 20, 50, dan 100 individu. Pengujian ini bertujuan untuk mengevaluasi pengaruh ukuran populasi terhadap nilai fungsi objektif dan waktu komputasi.

Konfigurasi ukuran populasi 50 dan jumlah generasi 100 digunakan pada kedua kelompok eksperimen sebagai titik pembanding. Dengan demikian, terdapat enam konfigurasi pengujian Genetic Algorithm yang dijalankan, tetapi hanya lima kombinasi parameter yang unik. Karena setiap konfigurasi diuji sebanyak tiga kali, total pengujian Genetic Algorithm adalah 18 *run*.

Sementara itu, Hill Climbing dan Simulated Annealing masing-masing diuji sebanyak tiga kali. Dengan demikian, keseluruhan eksperimen terdiri atas 24 run, yaitu 3 run Hill Climbing, 3 run Simulated Annealing, dan 18 run Genetic Algorithm.

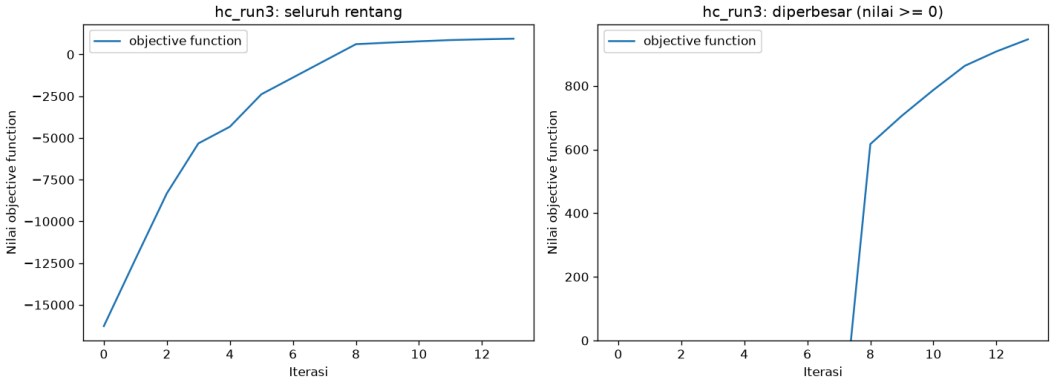
Hasil pengujian selanjutnya dianalisis menggunakan nilai fungsi objektif, rata-rata waktu eksekusi, serta validitas solusi akhir. Grafik perkembangan fungsi objektif dan visualisasi tiga dimensi juga digunakan untuk mengamati proses pencarian serta membandingkan konfigurasi paket sebelum dan sesudah optimasi.

II.5.2 Hasil Pengujian Hill Climbing

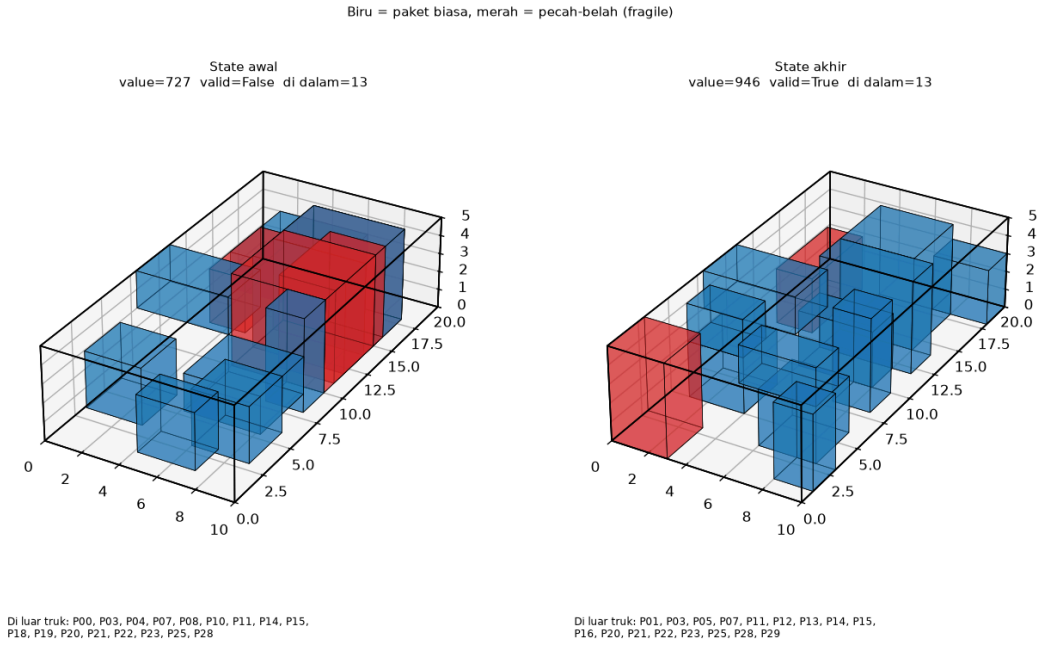
Tabel II.2 Hasil Pengujian Hill Climbing

Run	Final Score	Iterasi	Durasi(s)	Valid
1	1054	19	0.83	True
2	962	14	0.63	True
3	946	13	0.56	True

Gambar II.1 Grafik Nilai Objek Hill Climbing



Gambar II.2 Visualisasi State Awal & Akhir Hill Climb



Berdasarkan hasil pengujian, algoritma Hill Climbing menghasilkan nilai fungsi objektif sebesar 1054, 962, dan 946, dengan rata-rata sebesar 987,33. Seluruh pengujian menghasilkan konfigurasi akhir yang valid.

Jumlah iterasi yang diperlukan relatif sedikit, yaitu antara 13 hingga 19 iterasi, dengan rata-rata durasi eksekusi sekitar 0,67 detik. Hal ini menunjukkan bahwa algoritma mampu meningkatkan kualitas solusi dalam sejumlah perpindahan yang relatif terbatas.

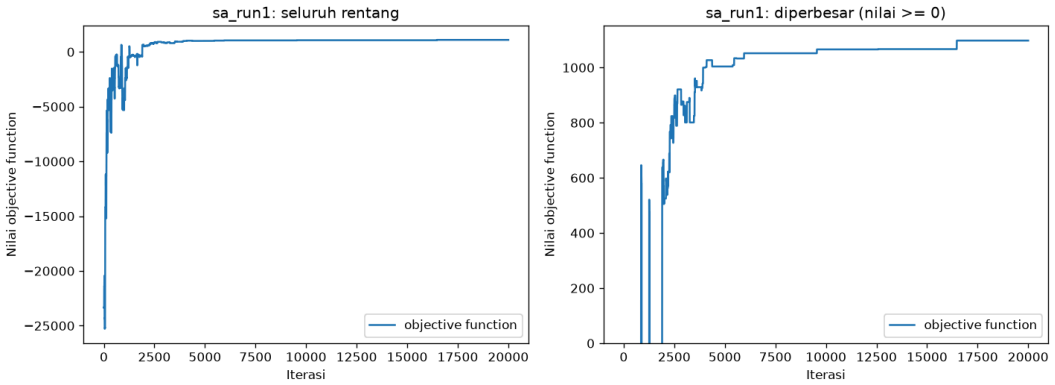
Meskipun demikian, perbedaan nilai akhir pada ketiga pengujian menunjukkan bahwa hasil pencarian dipengaruhi oleh state awal dan proses pembangkitan neighbor. Algoritma dapat berhenti pada local optimum yang berbeda karena hanya menerima perpindahan yang meningkatkan nilai fungsi objektif.

II.5.3 Hasil Pengujian Simulated Annealing

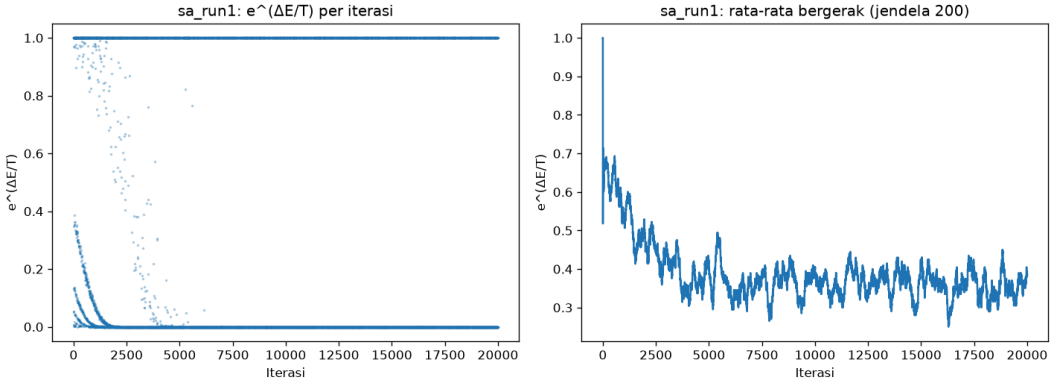
Tabel II.3 Hasil Pengujian Simulated Annealing

Run	Final Score	Stuck Count	Durasi(s)	Valid
1	1098	9	1.91	True
2	1082	9	1.87	True
3	1092	8	1.86	True

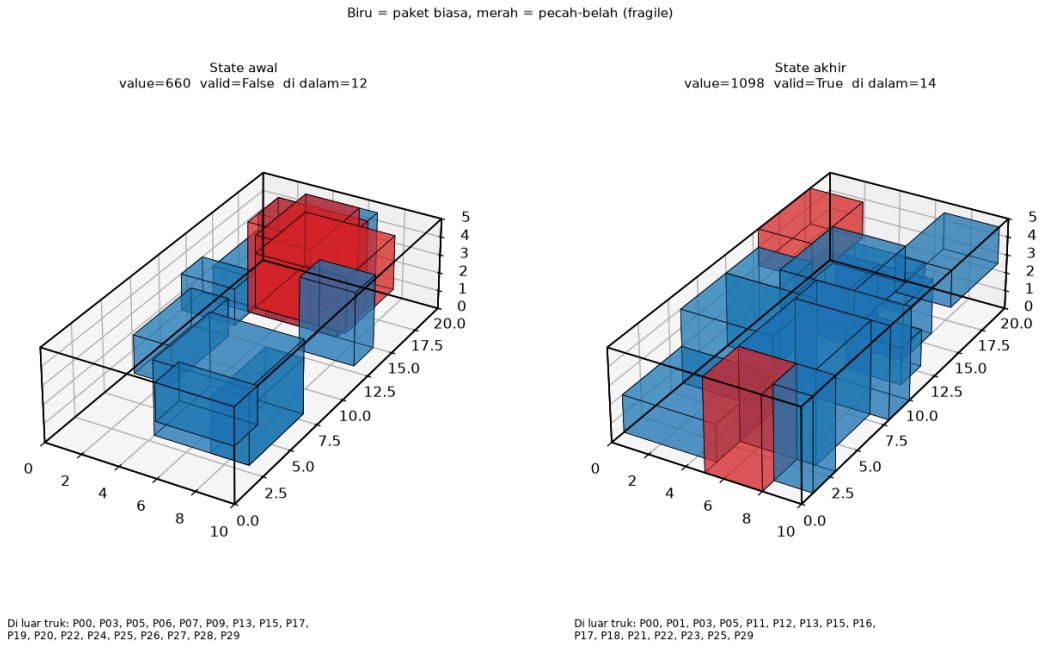
Gambar II.3 Grafik Nilai Objektif Simulated Annealing



Gambar II.4 Grafik Probabilitas Penerimaan Simulated Annealing



Gambar II.5 Visualisasi State Awal & Akhir Simulated Annealing



Berdasarkan hasil pengujian, Simulated Annealing menghasilkan nilai fungsi objektif sebesar 1098, 1082, dan 1092, dengan rata-rata sebesar 1090,67. Ketiga konfigurasi akhir yang diperoleh dinyatakan valid.

Nilai tertinggi diperoleh pada pengujian pertama, yaitu sebesar 1098, sedangkan nilai terendah sebesar 1082. Selisih nilai akhir yang relatif kecil menunjukkan bahwa Simulated Annealing menghasilkan kualitas solusi yang cukup konsisten pada tiga pengujian tersebut.

Rata-rata waktu eksekusi yang diperlukan adalah 1,88 detik, lebih tinggi dibandingkan Hill Climbing. Hal ini berkaitan dengan jumlah iterasi yang ditetapkan sebanyak 20000 iterasi.

Berdasarkan grafik perubahan fungsi objektif, peningkatan nilai terbesar terjadi pada tahap awal pencarian. Setelah beberapa ribu iterasi, nilai fungsi objektif selama proses pencarian cenderung stabil, meskipun masih mengalami fluktuasi akibat mekanisme penerimaan neighbor. Hal tersebut menunjukkan bahwa sebagian besar peningkatan kualitas solusi telah diperoleh sebelum seluruh iterasi selesai.

Grafik probabilitas penerimaan juga memperlihatkan bahwa kemungkinan menerima perpindahan dengan skor lebih rendah bergantung pada perubahan nilai fungsi objektif dan temperatur. Seiring berjalannya iterasi, penurunan temperatur membatasi penerimaan perpindahan yang merugikan.

Secara keseluruhan, Simulated Annealing menghasilkan rata-rata nilai fungsi objektif tertinggi pada eksperimen ini, meskipun membutuhkan waktu eksekusi yang lebih lama.

II.5.4 Hasil Pengujian Genetic Alorithm

Tabel II.4 Hasil Pengujian Genetic Algorithm Dengan Populasi Tetap

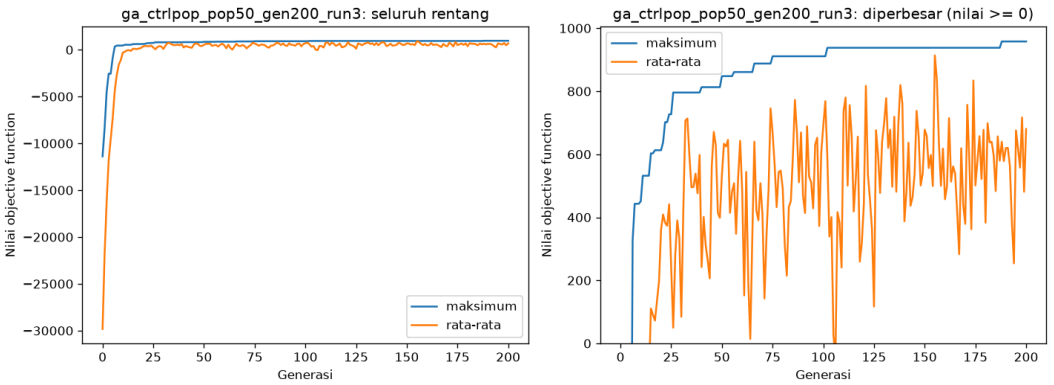
Populasi	Generasi	Rata-Rata FInal Score	Rata-Rata Durasi(s)	Valid
50	50	889.7	0.15	3/3
50	100	947	0.35	3/3
50	200	979.3	0.8	3/3

Tabel II.5 Hasil Pengujian Genetic Algorithm Dengan Generasi Tetap

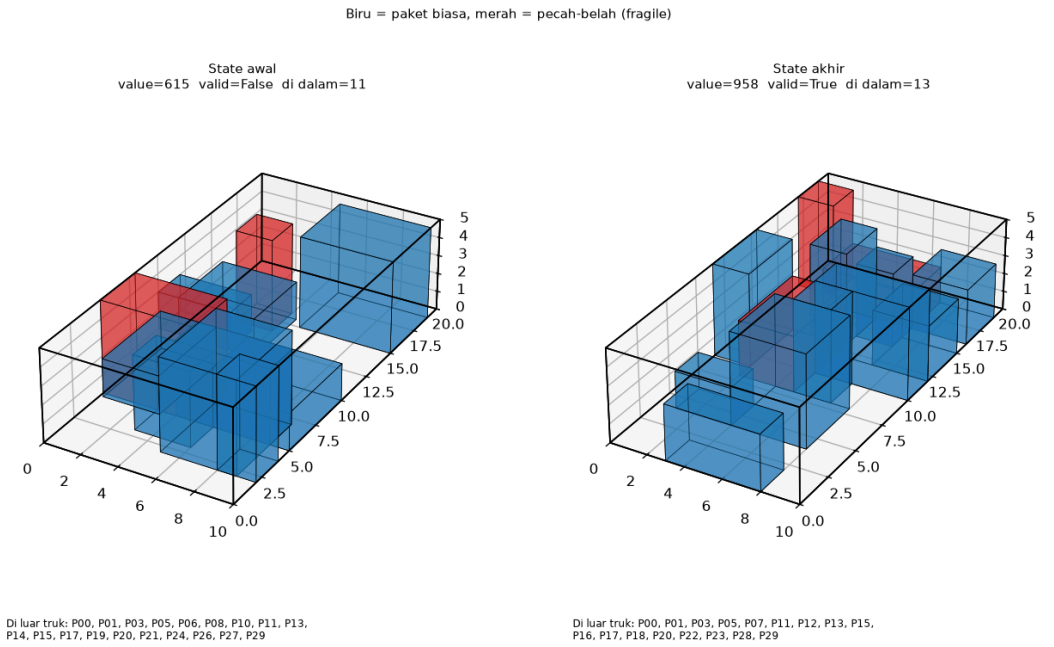
Populasi	Generasi	Rata-Rata FInal Score	Rata-Rata Durasi(s)	Valid
20	100	926.7	0.14	3/3

50	100	947	0.37	3/3
100	100	989.7	0.78	3/3

Gambar II.6 Grafik Perkembangan Nilai Fitness Maksimum dan Rata-rata pada Genetic Algorithm



Gambar II.7 Visualisasi State Awal & Akhir Genetic Algorithm



Pada pengujian Genetic Algorithm dengan ukuran populasi tetap sebesar 50, peningkatan jumlah generasi dari 50 menjadi 100 meningkatkan rata-rata nilai fungsi objektif dari 889,7 menjadi 947. Ketika jumlah generasi ditingkatkan menjadi 200, rata-rata nilai fungsi objektif meningkat kembali menjadi 979,3.

Peningkatan jumlah generasi tersebut juga diikuti peningkatan rata-rata waktu eksekusi dari 0,15 detik menjadi 0,35 detik dan kemudian 0,80 detik. Hal ini menunjukkan adanya pertukaran antara kualitas solusi dan kebutuhan waktu komputasi.

Pada pengujian dengan jumlah generasi tetap sebesar 100, peningkatan ukuran populasi dari 20 menjadi 50 meningkatkan rata-rata nilai fungsi objektif dari 926,7 menjadi 947. Ketika ukuran populasi diperbesar menjadi 100, rata-rata nilai fungsi objektif kembali meningkat menjadi 989,7.

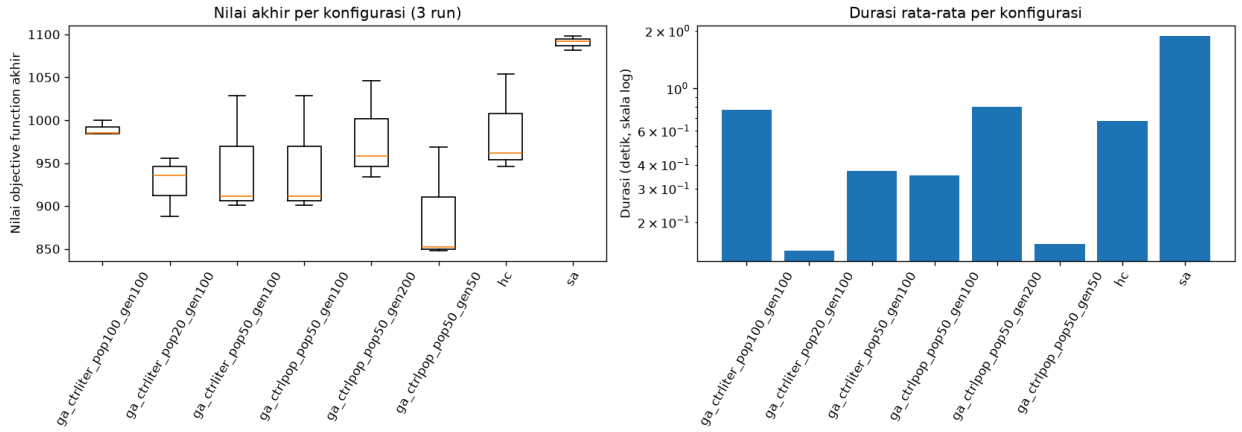
Peningkatan ukuran populasi juga menyebabkan bertambahnya durasi eksekusi, yaitu dari 0,14 detik pada populasi 20 menjadi 0,78 detik pada populasi 100.

Berdasarkan grafik Genetic Algorithm, nilai fitness maksimum cenderung meningkat dan kemudian stabil. Sementara itu, nilai rata-rata fitness populasi mengalami fluktuasi karena adanya proses crossover dan mutasi yang dapat menghasilkan individu dengan kualitas lebih rendah.

Maka dari itu, peningkatan jumlah generasi maupun ukuran populasi memberikan perbaikan nilai fungsi objektif pada konfigurasi yang diuji, tetapi juga meningkatkan waktu komputasi.

II.5.5 Perbandingan Algoritma

Gambar II.8 Grafik Perbandingan Nilai Fungsi Objektif dan Durasi Eksekusi



Tabel II.6 Tabel Perbandingan Hasil Algoritma

Algoritma	Rata-rata skor	Rata-Rata Durasi(S)	Valid
Hill Climbing	987.33	0.67	3/3
Simulated Annealing	1090.7	1.88	3/3
GA(Pop. 50, Gen. 100)	947	0.35	3/3

Berdasarkan hasil uji, Simulated Annealing menghasilkan rata-rata nilai fungsi objektif tertinggi sebesar 1090,67. Hill Climbing menghasilkan rata-rata sebesar 987,33, sedangkan Genetic Algorithm dengan konfigurasi populasi 50 dan 100 generasi menghasilkan rata-rata sebesar 947.

Dari segi kualitas solusi, Simulated Annealing menunjukkan hasil yang paling baik pada konfigurasi dan data pengujian yang digunakan. Selain menghasilkan nilai rata-rata tertinggi, ketiga pengujiannya juga menghasilkan konfigurasi valid dengan perbedaan nilai akhir yang relatif kecil.

Dari segi waktu eksekusi, Genetic Algorithm dengan populasi 20 dan 100 generasi memiliki rata-rata durasi terendah, yaitu 0,14 detik. Namun, rata-rata nilai fungsi objektif yang dihasilkan hanya sebesar 926,7. Sementara itu, Hill Climbing memiliki rata-rata durasi sekitar 0,67 detik dengan rata-rata nilai fungsi objektif sebesar 987,33.

Simulated Annealing memerlukan waktu rata-rata sekitar 1,88 detik, lebih lama dibandingkan kedua algoritma lainnya pada konfigurasi yang diuji. Meskipun demikian, algoritma tersebut menghasilkan rata-rata nilai fungsi objektif yang lebih tinggi.

Seluruh hasil pengujian yang dilaporkan menghasilkan konfigurasi akhir yang valid. maka dari itu, perbedaan performa ketiga algoritma sangat terlihat pada nilai fungsi objektif yang diperoleh dan waktu komputasi yang diperlukan.

Berdasarkan hasil tersebut, Simulated Annealing merupakan algoritma dengan kualitas solusi terbaik dalam eksperimen ini. Akan tetapi, untuk kebutuhan yang lebih mengutamakan waktu komputasi, Hill Climbing maupun konfigurasi tertentu dari Genetic Algorithm dapat menjadi alternatif.

BAB III

KESIMPULAN DAN SARAN

III.1 Kesimpulan

Berdasarkan implementasi dan hasil pengujian algoritma-algoritma di atas dalam menyelesaikan persoalan optimasi pemuatan paket pada Mejiro Express, diperoleh kesimpulan sebagai berikut.

- Ketiga algoritma berhasil diimplementasikan untuk menyelesaikan permasalahan optimasi pemuatan paket dengan mempertimbangkan dimensi truk, kapasitas berat, posisi paket, kemungkinan tumpang tindih, serta batasan terkait paket pecah-belah. Seluruh konfigurasi akhir yang dilaporkan pada eksperimen memenuhi constraint yang ditentukan.
- Algoritma Steepest-Ascent Hill Climbing menghasilkan rata-rata nilai fungsi objektif sebesar 987,33 dengan rata-rata waktu eksekusi sekitar 0,67 detik. Algoritma ini dapat meningkatkan kualitas solusi dalam jumlah iterasi yang relatif sedikit, tapi memiliki keterbatasan karena hanya menerima perpindahan menuju solusi yang lebih baik sehingga berpotensi terjebak pada local optimum.
- Algoritma Simulated Annealing menghasilkan rata-rata nilai fungsi objektif tertinggi, yaitu sebesar 1090,67, dengan rata-rata waktu eksekusi sekitar 1,88 detik. Hasil ini menunjukkan bahwa mekanisme penerimaan solusi yang lebih buruk berdasarkan probabilitas dapat membantu algoritma mengeksplorasi ruang pencarian dan memperoleh solusi berkualitas lebih tinggi pada eksperimen yang dilakukan.
- Pada Genetic Algorithm, peningkatan ukuran populasi maupun jumlah generasi menghasilkan peningkatan rata-rata nilai fungsi objektif dalam konfigurasi yang diuji. Konfigurasi dengan populasi 100 dan 100 generasi menghasilkan rata-rata nilai fungsi objektif sebesar 989,7, sedangkan konfigurasi populasi 50 dan 200 generasi menghasilkan rata-rata sebesar 979,3. Namun, peningkatan parameter tersebut juga disertai bertambahnya waktu komputasi.

- Berdasarkan perbandingan kualitas solusi, Simulated Annealing merupakan algoritma dengan hasil terbaik pada eksperimen ini. Sementara itu, Genetic Algorithm dengan konfigurasi populasi 20 dan 100 generasi menghasilkan rata-rata waktu eksekusi terendah sebesar 0,14 detik, tetapi dengan nilai fungsi objektif yang lebih rendah.
- Pemilihan algoritma yang sesuai perlu mempertimbangkan keseimbangan antara kualitas solusi dan waktu eksekusi. Algoritma dengan nilai fungsi objektif tertinggi belum tentu menjadi pilihan paling efisien apabila waktu komputasi merupakan prioritas utama.

III.2 Saran

Berdasarkan hasil implementasi dan pengujian yang dilakukan, terdapat beberapa saran untuk pengembangan program lebih lanjut.

- Proses pembangkitan neighbor dapat ditingkatkan dengan mempertimbangkan kemungkinan pelanggaran constraint sebelum suatu perubahan diterapkan. Dengan demikian, jumlah kandidat yang tidak valid dapat dikurangi dan proses pencarian berpotensi menjadi lebih efisien.
- Optimasi parameter Simulated Annealing. Mengingat algoritma ini menghasilkan kualitas solusi tertinggi tetapi membutuhkan waktu eksekusi relatif lebih lama, pengujian selanjutnya dapat dilakukan dengan memvariasikan temperatur awal, faktor pendinginan, dan batas jumlah iterasi untuk memperoleh kombinasi parameter yang lebih efisien.
- Pengembangan Genetic Algorithm. Pengujian dapat diperluas dengan memvariasikan probabilitas mutasi, metode crossover, serta mekanisme seleksi individu. Selain itu, penerapan mekanisme perbaikan solusi setelah crossover dan mutasi dapat dipertimbangkan untuk meningkatkan kemungkinan terbentuknya konfigurasi yang valid.
- Penyempurnaan fungsi objektif. Penggunaan penalti tetap untuk seluruh jenis pelanggaran dapat dikembangkan menjadi sistem penalti berbobot atau adaptif agar proses pencarian dapat membedakan tingkat pelanggaran dan lebih efektif mencapai solusi yang valid.

- Pengujian dengan variasi data yang lebih luas. Penelitian selanjutnya dapat menggunakan jumlah paket, dimensi truk, kapasitas maksimum, dan karakteristik paket yang berbeda. Hal ini diperlukan untuk mengevaluasi konsistensi hasil dan skalabilitas setiap algoritma pada permasalahan yang lebih kompleks.
- Peningkatan metode evaluasi. Jumlah pengulangan eksperimen dapat ditingkatkan untuk memperoleh estimasi performa yang lebih stabil. Selain itu, penggunaan batas waktu atau jumlah evaluasi fungsi objektif yang setara dapat dipertimbangkan agar perbandingan efisiensi algoritma menjadi lebih adil.

BAB IV

PEMBAGIAN TUGAS

Pembagian tugas tiap anggota kelompok diuraikan pada Tabel IV.1.

Tabel IV.1 Pembagian Tugas

NIM	Nama	Pembagian Tugas
13521154	Naufal Baldemar Ardanni	Konfigurasi dan struktur repositori, pengujian, perbaikan algoritma, perbaikan dan penambahan visualisasi, batas atas optimum, README, serta laporan
13524111	Reynard Anderson Wijaya	Kelas Package, Truck, State, batasan (constraints.py), *objective function*, langkah *neighbor*, tiga algoritma (*hill-climbing*, *simulated annealing*, *genetic algorithm*), pembangkit data, skrip eksperimen (demo.py), visualisasi awal, serta Pengisian Laporan Bab II
13524117	Rainaldi Pratama F Sembiring	Pengujian dan pelengkapan laporan

LAMPIRAN

1. **Tautan Repositori GitHub**

   https://github.com/NaufalArdanni/IF3170_G39_LocalKirkification

2. Surat Pernyataan Penggunaan AI

Dengan ini, kami

nama : Kelompok LocalKirkification (39)

1. Naufal Baldemar Ardanni (13521154)
2. Reynard Anderson Wijaya (13524111)
3. Rainaldi Pratama F Sembiring (13524117)

fakultas/sekolah : Sekolah Teknik Elektro dan Informatika (STEI)

kode mata kuliah : IF3170

nama mata kuliah : Inteligensi Artifisial (*Artificial Intelligence*)

judul tugas : Tugas Besar 1 (*Searching*)

menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas. Alat kecerdasan buatan (AI) yang digunakan adalah sebagaimana tercantum pada Tabel L.2.

Tabel L.2 Penggunaan AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan *tools* seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan *tools* seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan *tools* seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan *tools* seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya	Cara pengisian parameter agar visualisasi kotak (paket dan kontainer) sesuai harapan.

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa tiap penggunaan kecerdasan buatan (AI) yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Naufal Baldemar Ardanni

NIM 13521154



Reynard Anderson Wijaya

NIM 13524111



Rainaldi Pratama F Sembiring

NIM 13524117

REFERENSI

Tim Dosen IF3170. 2026. *IF3170 Inteligensi Artifisial: Beyond Classical Search*. Bandung: Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2020. [Video]. Available: Beyond Classical Search: Classical vs Local Search. [Accessed: Sep. 22, 2026].

M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2020. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM [Accessed: Sep. 22, 2026].

M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2020. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 22, 2026].

M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2020. [Video]. Available: Beyond Classical Search: Simulated Annealing. [Accessed: Sep. 22, 2026].

M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2020. [Video]. Available: Beyond Classical Search: Genetic Algorithm. [Accessed: Sep. 22, 2026].

Python Software Foundation, “itertools – Functions creating iterators for efficient looping,” Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/itertools.html. [Accessed: Oct. 15, 2026].

Python Software Foundation, “json – JSON encoder and decoder,” Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/json.html. [Accessed: Oct. 15, 2026].

Matplotlib Development Team, “matplotlib.pyplot,” Matplotlib 3.5.3 Documentation. [Online]. Available: https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html. [Accessed: Oct. 17, 2026].

Matplotlib Development Team, “Poly3DCollection,” Matplotlib Documentation. [Online]. Available: https://matplotlib.org/stable/api/_as_gen/mpl_toolkits.mplot3d.art3d.Poly3DCollection.html. [Accessed: Oct. 17, 2026].